

HyperParallel: 一种超节点亲和性 AI 框架*

Xin Zhang[†], Beilei Sun[‡], Teng Su[‡], Qinghua Zhang[‡], Chong Bao[‡],
Lei Chen[†], and Xuefeng Jin[‡]

[†] 香港科技大学 & 香港科技大学 (广州), 香港及广州, 中国

[‡] 华为技术有限公司, 中国

Abstract

大规模、稀疏、多模态和智能体 AI 模型的涌现, 与硬件向超节点架构的转变同步发生。超节点架构将数百乃至数千个加速器通过超低延迟互联和统一内存池集成在一起。然而, 现有的 AI 框架并未设计为高效利用这些架构, 导致编程复杂度高、负载不均衡以及内存利用率低等问题。本文提出了一种超节点亲和性 AI 框架, 将超节点视为单一逻辑计算机, 并将硬件感知的编排嵌入到框架中。在 MindSpore 中实现的 HyperParallel 架构包含三个核心组件: 用于自动分层内存管理的 HyperOffload、用于跨异构工作负载细粒度 MPMD 并行的 HyperMPMD, 以及用于声明式并行策略规范的 HyperShard。这些技术协同作用, 在降低并行编程和系统调优开销的同时, 显著提升了训练和推理效率, 证明了超节点亲和性对下一代 AI 框架的必要性。

1 引言

人工智能的快速进步由模型架构、训练范式和硬件系统的协同演进推动 [14, 15, 16]。现代基础模型在参数规模 [6, 7, 20] 和功能范围上持续扩展, 从纯语言系统扩展到多模态 [22]、稀疏 [3, 5] 和智能体架构 [24]。这些模型在计算负载上引入了显著的异构性、不规则的数据路由, 以及对内存和通信带宽的海量需求。与此同时, 硬件架构已演进为超节点集成集群——数千个加速器通过超低延迟的点对点互联和统一内存池相连 [10, 11, 26]。在这一背景下, AI 框架作为关键的中间件层, 成为连接高层算法抽象与日益复杂的硬件架构的桥梁。

我们发现, 现有 AI 框架面临着性能-可编程性鸿沟。传统的单程序多数据 (SPMD) 范式 [4] 难以处理专家混合 (MoE) [3, 5] 和多模态架构固有的负载不均衡问题。此外, 跨分层内存层手动管理中间状态 (权重、激活值和 KV 缓存) 给研究人员带来了沉重的工程负担。现有的系统扩展层 [1, 2, 8, 9, 18, 19, 21] 虽在特定领域有效, 但往往导致模型逻辑与硬件拓扑的紧密耦合, 代码可移植性差, 集群利用率也不理想。

这些挑战表明, 仅对现有框架进行增量优化已不足够。相反, AI 框架必须经历一次以明确拥抱超节点特性、重新思考并行性、内存和执行抽象为核心的架构转变。本文主张, 超节点亲和性应成为下一代 AI 框架的核心设计原则。超节点亲和性框架将超节点视为单一的、逻辑统一的计算实体, 并将硬件感知的编排直接嵌入框架运行时和编译器中。这样既能屏蔽算法开发者的底层系统复杂性, 又能系统性地挖掘现代硬件的性能潜力。

基于这一洞见, 我们提出了超节点亲和性 AI 框架, 并在 MindSpore 中展示了其设计与实现。该框架引入了新颖的 HyperParallel 架构, 整合三个核心组件: HyperOffload——通过统一内存池和自动卸载实现计算与模型状态的解耦; HyperMPMD——将并行执行从 SPMD 扩展到细粒度的多程序多数据 (MPMD), 以处理异构的多任务工作负载; HyperShard——提供将算法逻辑与并行策略设计解耦的声明式并行编程接口。这些组件共同为超节点集群上的 MoE、全模态和强化学习工作负载提供灵活、高效的执行能力。

本文做出以下贡献:

* 贡献者完整列表见论文末尾的致谢章节。

- 新兴挑战的系统性分析：我们分析了模型工作负载、硬件架构和训练-推理范式的最新趋势，并识别了它们在超节点部署时对现有 AI 框架的根本性限制。
- 超节点亲和性框架设计：我们提出了一种新的架构范式，将超节点视为“巨型计算机”，在框架内抽象复杂的网络拓扑、分层内存系统和异构执行环境。
- HyperParallel 架构：我们引入并实现了 HyperOffload、HyperMPMD 和 HyperShard，实现统一的状态管理、灵活的 MPMD 并行以及声明式并行编程。
- 实证验证：我们证明了所提框架在代表性大规模工作负载上显著提升了训练和推理效率，同时降低了开发和优化开销，验证了其有效性。

本文其余部分组织如下：第 2 节刻画了模型和硬件的演进，回顾了现有 AI 框架，并分析了万亿参数 LLM 时代 AI 框架所面临的挑战。第 3 节介绍了所提超节点亲和性框架的设计和关键技术。第 4 节以对结果的讨论和未来方向作结。

2 背景

AI 框架作为关键的中间件层，位于模型算法和底层硬件架构之间，是加速整个人工智能领域进步的基础软件基础设施。其演进本质上由算法复杂性和硬件创新的双重需求驱动。一方面，框架必须提供高层次抽象，使研究人员能够高效地开发、调试和验证新颖的架构，从而促进快速的算法迭代。另一方面，它必须系统地将这些高层次抽象转化为最大化硬件利用率、充分挖掘专用计算资源潜力的优化可执行程序。

2.1 小型深度学习模型时代

模型与硬件。在深度学习的早期阶段，大多数模型体量较小。例如，小规模计算机视觉（CV）模型 [25] 通常可以放入单个加速器的内存中。参数量适中的自然语言处理（NLP）模型 [13] 虽然超出了单卡内存容量，但通常可以在具有多个 GPU 的单台机器上运行。

训练范式。训练小型深度学习模型时，CV 模型的主要训练策略是数据并行（DP），单卡计算能力是主要性能瓶颈。对于中等规模的 NLP 模型，通常采用模型并行将组件分片到节点内的多张卡上。较高的节点内通信量使内部带宽成为关键瓶颈。

AI 框架。为了在单台机器上处理此类工作负载，PyTorch [16] 凭借高度的执行灵活性成为主导生态系统。PyTorch 的核心优势在于其动态计算图和丰富的生态系统，使其非常适合快速原型设计、模型定制和学术研究。

2.2 十亿规模大型语言模型时代

模型与硬件。随着大型语言模型的出现，模型参数量迅速扩展到数千亿。海量预训练数据集对硬件也提出了更高要求。通常在训练十亿规模 LLM 时需要构建包含多节点的服务器集群。

训练范式。预训练和监督微调是十亿规模 LLM 最常见的两种训练任务。多维分片（数据并行、张量并行和流水线并行）策略被广泛采用。不同的分片策略对集群带宽提出了不同的需求。特别是张量并行（TP）、流水线并行（PP）和上下文并行（CP）需要频繁的跨服务器通信——每次传输通常超过数 GB——这很难通过计算-通信重叠来隐藏。例如，在典型的训练设置中，TP 的数据流量开销占训练时间的 52.9% [11]。因此，跨机通信成为主要性能瓶颈，导致集群资源利用率不理想。

AI 框架。PyTorch 的主要局限性包括缺乏全面的多程序多数据（MPMD）能力，以及缺乏对计算与存储解聚的原生支持。为解决这些瓶颈，许多框架应运而生，如 JAX [2]、DeepSpeed [18]、Megatron [21] 和 HLX [9]，作为专用系统扩展层。这些框架专注于分布式策略、内存优化、并行拓扑、混合注意力兼容性和高效多模态处理。

- JAX：函数式编程与编译器驱动的加速 – JAX 利用函数式编程范式结合可组合的变换（jit, pmap, vmap），使编译器能够跨算子、并行性和微分执行统一优化。它在大规模 TPU 训练、科学计算以及需要精确控制计算图变换的研究中表现出色。然而，其生态系统高度专业化，导致编程复杂度较高。
- DeepSpeed：大规模训练的系统工具箱 – DeepSpeed 提供 ZeRO 系列冗余消除分片、MoE 优化器、高效通信和卸载支持，是在受限 GPU 资源下训练超大模型的关键基础设施。它作为系统增强层与 PyTorch 紧密集成，以弥补内存效率和分布式调度方面的不足。一个重大缺陷是其假设网络拓扑相

对均匀；而在实际超节点中，机架间延迟和带宽差异极大，DeepSpeed 对 SPMD 和静态分片的依赖可能导致严重的负载不均衡和硬件利用率低下。

- **Megatron：密集与稀疏模型的极致并行** – Megatron 强调流水线并行、张量并行和 MoE 并行的优化，以实现峰值的分布式性能。其“规模优先”的设计哲学非常适合在 GPU 资源充足的环境中追求最大吞吐量。尽管 Megatron-Core 正在演进为 LLM 训练的基础模块，但该框架存在通信逻辑与硬件拓扑紧密耦合的问题，导致代码可移植性差、编程复杂度高，参数调优和性能优化成本高昂。
- **HLX：混合注意力架构的加速** – HLX 专为具有线性和全局混合注意力的新型 Transformer 变体设计。它引入多线程并行编排和双维（行/列）计算流水线，以加速稀疏和线性注意力算子。通过深度整合算子结构与线程调度，HLX 为混合注意力模型提供了系统级加速方案。然而，它缺乏框架级的超节点统一内存池管理，迫使研究人员手动管理训练和推理的中间状态。

相关研究工作。除上述框架外，还有一些研究工作解决了 LLM 预训练整个生命周期中的其他瓶颈。

- **集群级容错与算法-系统协同设计** – MegaScale [8] 代表了集群级训练基础设施的发展趋势。在跨多节点的大规模分布式算力环境中，它通过算法/内核优化、3D 并行通信重叠和自动故障监控，在万卡集群上实现了较高的模型 FLOP 利用率 (MFU)。然而，实现有效的计算-通信重叠需要高度定制的并行策略和代码实现，导致调试和迁移成本极高。
- **中大规模训练的分片与卸载** – 以 ZeRO 生态系统（特别是 ZeRO-3 和 ZeRO-Offload）[17, 19] 为代表，该方向针对数百亿到数万亿参数的模型训练。核心方法是在 GPU 集群间对参数、梯度和优化器状态进行全分片，通常结合 CPU/NVMe 卸载来打破 GPU 内存墙。这些方法利用高效的通信分区实现近线性扩展。然而，它们依赖静态内存分区，缺乏统一内存池的自动管理能力，可能导致内存碎片化。
- **超大规模加速器集群的调度与通用执行层** – Pathways [1] 展示了超大规模集群统一执行层的必要性。该方向专注于跨加速器 Pod 的异步数据流、调度和拓扑优化，以实现流水线并行、数据并行和模型并行之间的无缝协作。与传统的刚性 SPMD 模型不同，它支持 MoE 模型所必需的细粒度、不规则路由。然而，其对中心化资源管理器和协调器的依赖，可能在跨异构节点管理高度动态的大规模任务图时引入调度开销和延迟瓶颈。

2.3 万亿规模大型语言模型时代

模型。近期，主流大型语言模型在规模、架构和目标模态上出现了重大变化。具体而言，最先进的 LLM 通常拥有万亿级参数，并利用专家混合（MoE）架构在扩展参数的同时增加稀疏性，对文本以外多种模态的支持也日益普及。

- 从模型规模和架构角度看，许多最先进的 LLM，如 DeepSeek-V3 [12] 和 Qwen-3 [23]，利用 MoE 架构在扩展参数时增加稀疏性。这种方法在保持每 token 常数计算成本的同时扩展了模型容量并增强了泛化能力。
- 从纯语言模型到全模态系统的过渡在新发布的模型中日益普遍。这一过渡需要集成多样化的子模块，导致模型结构日趋不规则，使并行编程和优化都更加复杂。全模态模型通常采用多编码器、模态融合层、多解码器架构。多模态的桥接引入了模块级异构性和动态路由负载。

训练范式。训练范式从传统预训练转变为智能体强化学习（RL）。这一转变需要协同部署多种任务，包括训练、推理和智能体执行。这种异构化趋势对多任务、多模型工作流的编排提出了更严格的要求。具体地，为增强指令遵循、深度推理和复杂问题解决能力，最先进的大模型在初始预训练阶段后越来越多地引入 RL。强化学习将工作负载范式从纯静态数据集训练转变为涉及动态数据的“采样-评估-更新”异构任务。这一工作流通常依赖大规模的 rollout 操作和异步的 actor-learner 架构，训练和推理任务并发执行。因此，底层框架必须支持强健的异构任务管理、异步任务调度以及针对多任务、多模型环境定制的动态编排能力。

硬件特性。超节点集群专为训练具有 MoE、多模态和智能体特性的 LLM 而设计。以华为 Matrix384 [26] (Atlas 900) 超节点为例，我们分析超节点集群的典型特性：

- **统一内存寻址与点对点架构** – 基于 UB 互联协议（灵驱），系统将 384 个 Ascend 910C NPU 和 192 个鲲鹏 CPU 集成到单个超节点中。计算、内存和网络资源被解聚为独立的池化单元。
- **极高带宽与超低延迟** – 传统架构依赖 PCIe 或以太网进行互联。Ascend 384 超节点采用高效协议，与传统服务器架构相比通信带宽提升 15 倍，单跳延迟从 $2\mu\text{s}$ 降至 200ns——提升了十倍。

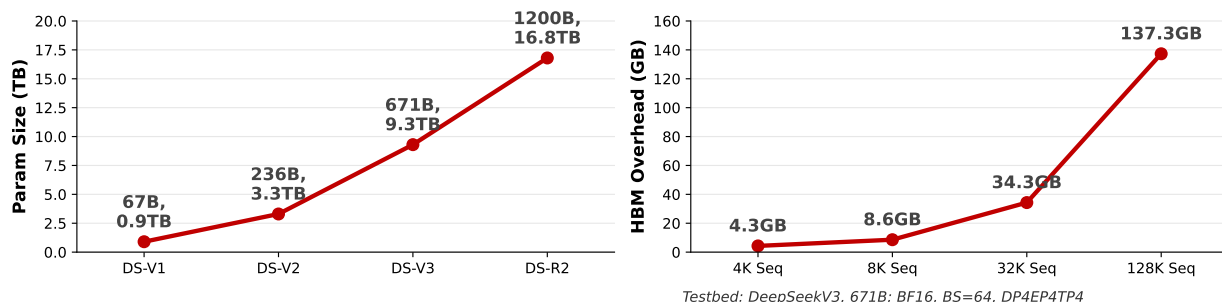


Figure 1: 在模型训练和推理过程中，存储和管理参数及中间状态的复杂性持续增加。

- 分层多样化的网络拓扑 – 单个 Ascend 超节点的规模预计将从 384 卡增加到 8192 卡，未来目标达到 15488 卡。为支持这种大规模可扩展互联，超节点采用分层拓扑：每个机架内建立 2D 全连接网络，然后跨机架扩展为另一个 2D 全连接网络，最终形成 4D 全互联（全网格）拓扑。

AI 框架面临的挑战。超节点为快速增长的模型参数和日益复杂的架构提供了优越的解决方案，但充分发挥其潜力需要应对若干根本性挑战。为降低算法开发和调优的复杂性，同时最大化大规模训练和推理效率，AI 框架面临以下关键挑战：

Table 1: 按模型类型的并行策略

模型 & 算法	策略
密集 Transformer	DP, PP, TP, SP
稀疏 MoE	DP, PP, TP, SP, EP
扩散模型	DP, FSDP
长序列	SP, CP
强化学习	MPMD
...	...

Table 2: 按集群类型的并行策略

集群	策略
单机 (8 DIE)	TP8, PP 负责其余
单机 (16 DIE)	高维 TP (TP16), 减少 PP
8k 节点超平面	拓扑感知 TP16, 减少 PP

- 挑战 1: 抽象复杂网络拓扑以简化并行编程 – 随着超节点消除节点间带宽瓶颈，集群规模可增长至 10 万卡级别。驾驭如此复杂的网络拓扑需要精心设计和优化的并行策略。模型架构或集群配置的任何变化都需要完全重新设计这些策略，如表 1 和表 2 的示例所示。经验上，每次适配和调优周期需要高级系统优化工程师 1-2 周的工作量，并消耗大量集群资源，导致运营效率低下。
- 挑战 2: 实现灵活细粒度的并行性以最大化硬件利用率 – 当前的并行分片主要依赖 SPMD 范式。然而，随着模型向 MoE 和全模态架构演进，不同模块和层呈现异构的计算负载。标准 SPMD 方法频繁导致负载不均衡；例如，在子模块负载各异的全模态模型中，SPMD 结合流水线并行 (PP) 往往引发显著的流水线气泡。此外，MoE 模型遭受高通信开销；在 DeepSeek-V3 中，专家并行 (EP) 通信占执行时间的 17%，通信掩盖率仅为 61%，远低于 90% 的理想目标。
- 挑战 3: 通过内存池化管理中间状态以优化内存利用率 – 随着模型规模从数千亿向数十万亿参数发展，管理中间状态（包括权重、激活值和 KV 缓存）的复杂性呈指数级增加。超节点通过内存语义互联提供统一的 DRAM 内存池。为保持性能，中间状态必须在使用前主动预取（换入）到高带宽存储器 (HBM) 中，以避免内存访问延迟。手动管理这些交换操作给算法研究人员带来了沉重的编程和优化负担。

3 超节点亲和性 AI 框架的设计

当前工业界训练和推理软件的优化工作不足以根本解决超节点对 AI 框架带来的系统性挑战。为充分利用超节点的优势并解决模型演进带来的瓶颈，AI 框架必须以超节点亲和性为核心进行架构重设计。通过整合超节点的特定架构特性，框架可以为研究人员提供变革性的编程体验，同时显著提升训练和推理效率。本章以 MindSpore 的实际实现为基础，介绍超节点亲和性 AI 框架的架构设计和关键技术。

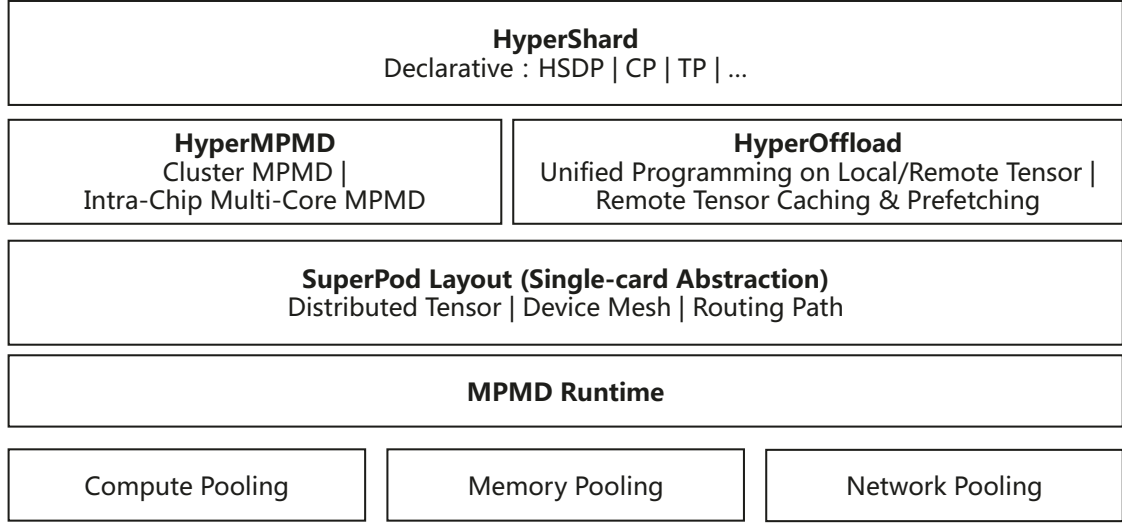


Figure 2: HyperParallel 总体设计。

3.1 总体设计

MindSpore 超节点亲和性的核心理念是通过 AI 框架抽象底层硬件复杂性，将超节点视为单一的“巨型计算机”。这种方法为算法研究人员提供了类似单节点系统的编程和执行体验。通过将复杂的软件工程任务（如并行分片和训练-推理状态管理）直接嵌入框架，MindSpore 利用超节点的点对点架构优势，解决 MoE、全模态和强化学习等复杂场景中的性能瓶颈。为此，我们提出如图 2 所示的 HyperParallel 架构，包含三个关键组件，旨在屏蔽用户的硬件复杂性，同时最大化架构优势：

- HyperOffload – 促进计算与状态的解耦。利用超节点的池化存储能力，解决不断增加的模型规模带来的内存瓶颈。
- HyperMPMD – 将并行分片从传统的 SPMD 范式过渡到细粒度的多程序多数据（MPMD）方法。利用点对点计算架构，为强化学习和全模态工作负载提供灵活的并行性。
- HyperShard – 推动从命令式到声明式编程的转变。它抽象超节点集群的复杂拓扑，为全模态和其他高级模型提供简约、可组合的编程体验。

在 MindSpore 上进行超节点模型开发、训练和推理的工作流程如下：

- 步骤 1：算法开发 – 研究人员通过框架开发算法，利用 HyperShard 的声明式编程接口配置模型的并行策略。
- 步骤 2：灵活并行 – HyperMPMD 以 MPMD 格式执行灵活、细粒度的并行，针对算法和集群的独特拓扑进行专门优化。
- 步骤 3：运行时编排 – 执行过程中，HyperOffload 对计算、存储和网络进行全局编排，管理 HBM 与 DRAM 之间的状态交换，以充分利用超节点的分层内存池。

3.2 HyperOffload

HyperOffload 的基本概念是将模型状态卸载到外部内存池，利用片上内存（HBM）作为高速缓存。这种架构允许单个加速器容纳显著更大的模型。核心挑战在于在不牺牲性能的情况下自动化 HBM 与 DRAM 之间的状态交换。为此，HyperOffload 采用多级缓存流水线调度和全局图编排，如图 3 所示。

- 多级缓存流水线调度 – HyperOffload 利用通信隐藏技术，在计算算子需要之前异步预取下一执行阶段所需的缓存块到高速存储层。通过将模型结构特性与数据访问模式预测相结合，系统动态调整预取路径，有效重叠加载延迟与计算时间。
- 全局图编排与调度 – HyperOffload 将分层缓存操作（如数据预取和块卸载）抽象为原生算子，实现缓存读写和迁移行为的统一描述。利用 MindSpore 的 JIT 图编译，框架在解析用户网络后自动插入

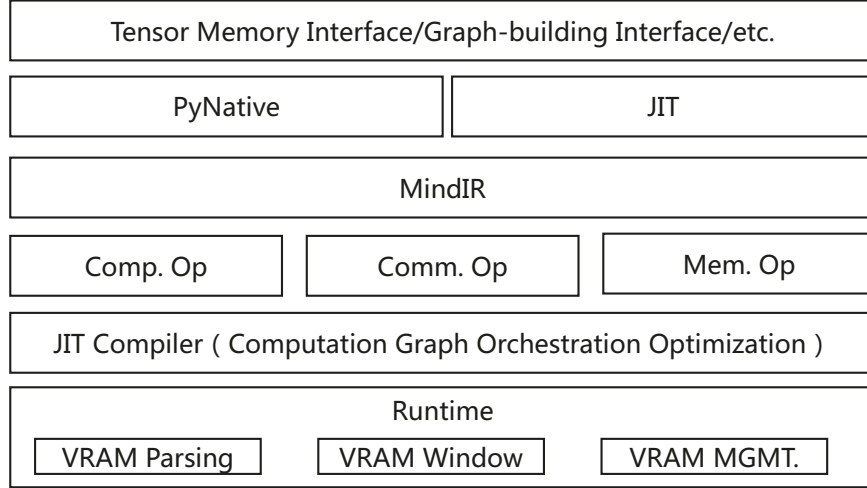


Figure 3: HyperOffload 架构。

缓存管理算子并重组执行流程。通过对缓存、计算和通信算子的统一编排，编译器自动调度并行执行链，消除了手动插入同步点的复杂性。

通过充分利用超节点的统一大内存池，HyperOffload 显著放宽了训练和推理中的 HBM 内存约束。这使得可以去除复杂的 N 维 SPMD 并行，改用简单的 1 维 SPMD 数据并行，减少了状态同步的通信开销，进一步提升了性能。

实证结果表明了显著改进：

- **训练场景** – 在相同硬件配置下，Llama-8B 模型每步迭代时间从传统方法的 5.2s 降至 4.08s，性能提升约 20%。
- **推理场景** – 在相同延迟限制下，HyperOffload 将支持的序列长度从 71K 提升至 123K，比传统方法提升 70%。

3.3 HyperMPMD

HyperMPMD 的核心目标是实现灵活、细粒度的多程序多数据 (MPMD) 分片方法，以解决全模态学习和强化学习等场景中的计算不均衡问题，从而最大化硬件利用率。HyperMPMD 在三个不同维度上提供 MPMD 能力：

- **子模型内 AICube 与 AIVector 级别的并发** – 通过对模型张量进行分片，并利用卡内 MPMD 调度实现 AICube 和 AIVector 任务，框架实现了计算-通信重叠的细粒度编排（如图 4(a) 所示）。这解决了 MoE 架构中固有的通信掩盖挑战。通过协同卡内多核并行与卡间并行，HyperMPMD 将通信掩盖率从传统的 60% 提升至 90%。
- **子模型间并发均衡** – 框架将子图解耦为独立的并发任务（如图 4(b) 所示），利用动态调度缓解负载不均衡。这有效消除了全模态或多模态模型中由异构子模块负载引起的 10%–40% 的流水线气泡，带来约 15% 的整体训练性能提升。
- **跨模型并发调度** – 与 MPMD 运行时集成，框架提供单一控制器 (Single Controller) 支持，在超节点池化计算资源内执行细粒度并行分片和动态调度（如图 4(c) 所示）。这实现了模型级并发，消除了落后者效应，解决了多任务强化学习中的负载不均衡，将全集群资源利用率提升了 15%。

HyperMPMD 根据模态或任务（如文本、图像、音频、融合和任务调度组）划分独立的 MPMD 进程组。每个组执行专门的程序逻辑，通过标准化接口进行通信。以下示例展示了全模态 MPMD 进程组的配置。通过将核心逻辑封装到独立模块中，并通过代码清单 1 中的配置文件定义节点到模块的映射，框架消除了刚性硬编码的需求。

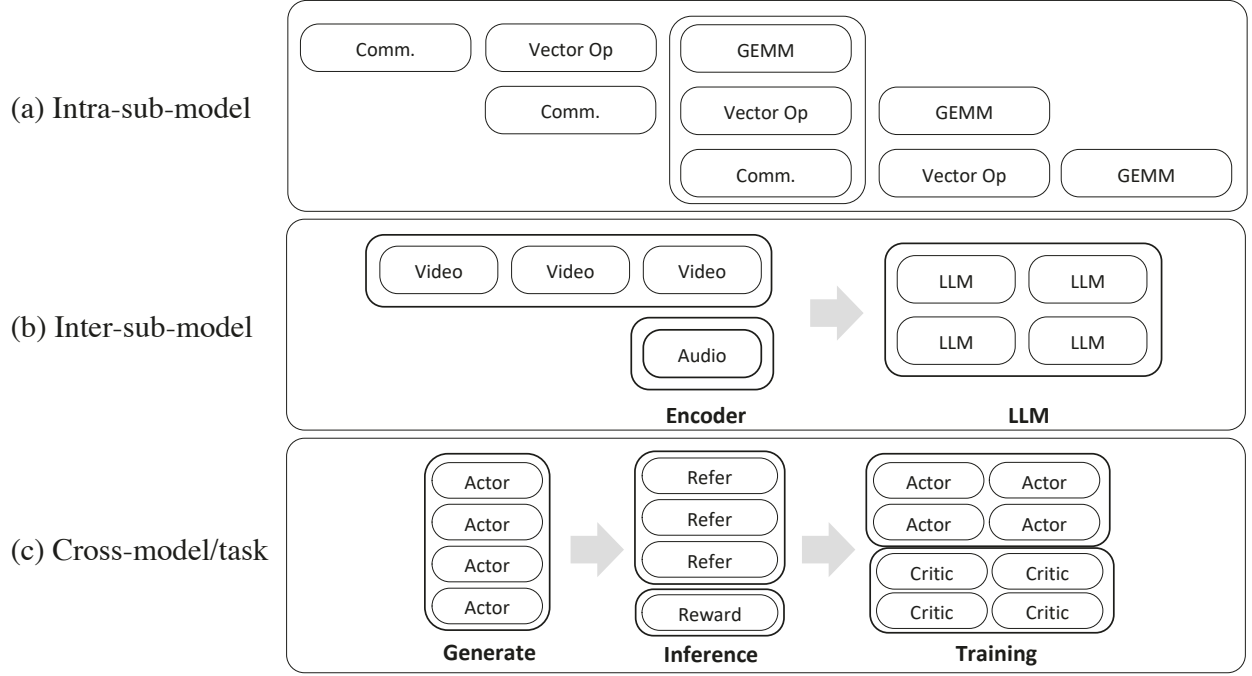


Figure 4: HyperMPMD 示意图。

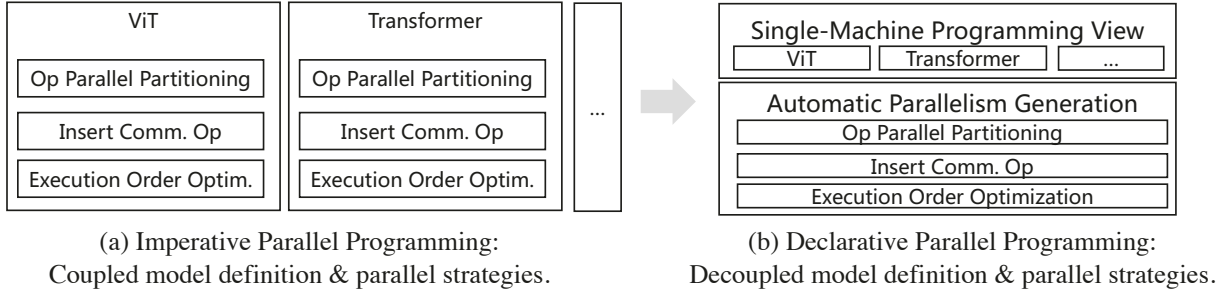


Figure 5: 声明式与命令式并行编程的比较。

Listing 1: 映射配置示例。

```

1 process_groups = {
2   "text": {"ranks": [0,1], "module": TextModule, "hardware": "NPU"},
3   "image": {"ranks": [2,3], "module": ImageModule, "hardware": "NPU"},
4   "audio": {"ranks": [4], "module": AudioModule, "hardware": "CPU"},
5   "fusion": {"ranks": [5], "module": CrossModalFusion, "hardware": "NPU"},
6   "task_scheduler": {"ranks": [6], "module": TaskPriorityScheduler,
7                     "hardware": "CPU"}
8 }

```

3.4 HyperShard

HyperShard 的基本目标是提供声明式并行编程能力，使开发者能够使用单节点编程规范构建模型，有效地将算法逻辑与并行策略解耦，允许后者根据网络拓扑灵活配置。如图 5(a) 所示，传统的命令式并行编程存在模型定义与并行策略的紧密耦合问题，需要针对每个模型变体进行定制的手动干预——如算子分片、显式通信算子插入和执行序列优化。相比之下，图 5(b) 展示了声明式范式：研究人员从单设备视角开发算法，仅声明所需的并行约束，框架随后自动生成底层并行策略，实现完全解耦。

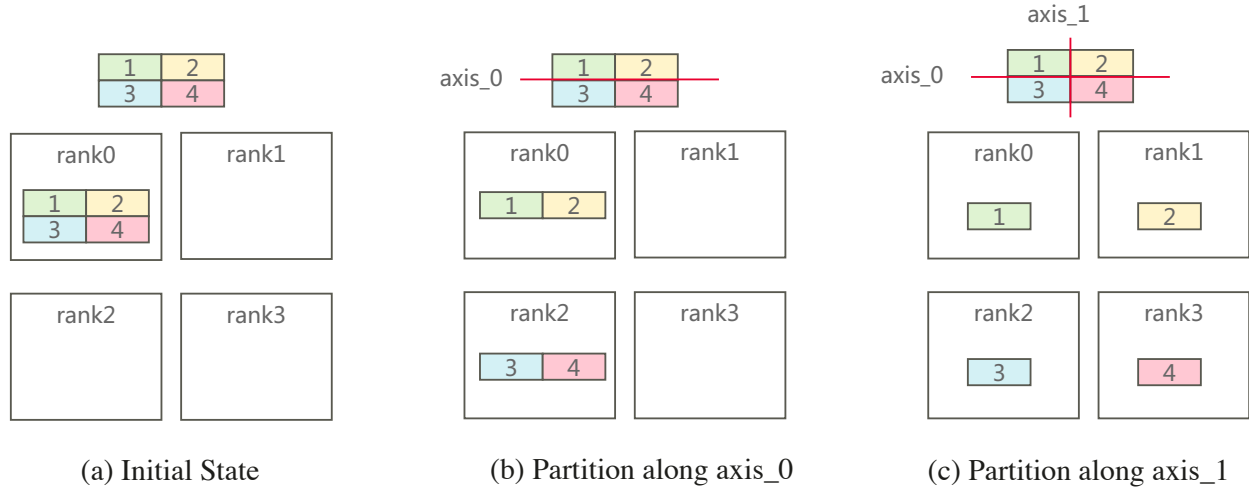


Figure 6: HyperShard 分区示意图。

HyperShard 的主要编程抽象是接口 `Layout(device_matrix, alias_name, tensor_map)`，定义如下：

- `device_matrix` – 描述集群内加速器的逻辑排列。
- `alias_name` – 为设备矩阵内的每个维度分配描述性标识符。
- `tensor_map` – 指定给定张量的每个维度如何在设备矩阵上进行分区。

Listing 2: HyperShard 策略生成示例。

```

1 # 4个加速器组成2*2设备矩阵
2 device_matrix = (2, 2)
3 # 设备矩阵每个维度的别名
4 alias_name = ("x", "y")
5 # 假设tensor.shape=(2, 2)
6 tensor_map = ("x", "y")
7 # 声明布局
8 layout = Layout(device_matrix, alias_name)
9 # 生成并行分区策略
10 parallel_strategy = layout(tensor_map)

```

以代码清单 2 中的代码为例，我们展示在八个加速器上分片张量的过程。自动分片过程分三个阶段进行：

- 图 6(a)：初始化 – 框架初始假设张量位于单个逻辑 rank（如 Rank 0）上。
- 图 6(b)：第一维度分区 – 由于 `tensor_map[0] = "x"`，框架沿 `device_matrix` 的 "x" 维度对张量的第 0 维进行分片。
- 图 6(c)：第二维度分区 – 由于 `tensor_map[1] = "y"`，框架沿 `device_matrix` 的 "y" 维度对张量的第 1 维进行分片。

关键需要注意的是，这一过程在编译时不涉及物理张量切片。相反，框架根据用户配置对并行策略 (`shard_strategy`) 进行形式化推导；实际的分片执行发生在运行时。

通过将并行策略与算法解耦，屏蔽开发者的硬件复杂性，HyperShard 显著降低了工程开销。新算法并行化所需的时间缩短至不到一天，并行策略优化周期从数天压缩至数小时。

4 总结与讨论

本文系统分析了模型工作负载、硬件架构和训练-推理范式的演进趋势，重点阐述了大规模模型和超节点集群对 AI 框架施加的挑战。我们识别了现有框架在并行分片、状态管理和集群利用率效率方面的关键不足，并提出了超节点亲和性 AI 框架的设计，基于 MindSpore 实现了 HyperParallel 架构。

通过三项核心技术——HyperOffload、HyperMPMD 和 HyperShard——的深度集成，框架实现了计算与状态的解耦、细粒度 MPMD 并行，以及声明式编程接口。这些创新带来以下贡献：

- 复杂拓扑的抽象 – 框架简化了开发和调优流程，使研究人员能够以与单节点编程相同的便利性开发大规模模型。
- 灵活高效的并行性 – 通过提供动态细粒度的并行策略调度，框架最大化了超节点计算资源的利用率，有效解决了 MoE、全模态和强化学习场景中的负载不均衡问题。
- 统一的状态管理 – 利用统一内存池管理中间状态，提高了内存利用率，支持不断增大的模型的训练和推理而不损失性能。

实证结果表明，在相同硬件条件下，HyperOffload 和 HyperMPMD 显著提升了训练和推理性能，同时大幅缩短了算法开发和并行策略优化所需的时间。展望未来，超节点亲和性 AI 框架可进一步扩展以支持异构任务调度、多节点动态扩展和跨数据中心协同训练，为下一代大规模模型研究和工业部署提供强大的软件基础设施。

5 致谢

我们诚挚感谢以下贡献者在 HyperParallel 框架开发中发挥的宝贵作用。

研究与工程

Changwei Ma, Chao Fu, Chen Li, Chongming Liu, Da Lei, Dawei Fan, Dongjie Geng, Fuxin Huang, Guangpeng Zhang, Haoran Wang, Huikang Tang, Jiahong Qian, Jiaqi Song, Jinshan Ding, Kaisheng Wang, Mingqi Li, Nuohang Li, Qi Guo, Qian Shu, Renwei Zhang, Shanni Li, Xiangyu Meng, Xinglei Xu, Xiong Gao, Yi Huang, Yide Wang, Yifan Yao, Yixing Feng, Zhenbang Wang, Zheng Li, Zhenzhang Yang, Zherui Chang, Zhibo Liang

References

- [1] Paul Barham et al. Pathways: Asynchronous distributed dataflow for ml. In Symposium on Operating Systems Design and Implementation (OSDI), 2022.
- [2] James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Nectou, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: composable transformations of Python+NumPy programs, 2018.
- [3] Zixiang Chen, Yihe Deng, Yue Wu, Quanquan Gu, and Yuanzhi Li. Towards understanding the mixture-of-experts layer in deep learning. Advances in neural information processing systems, 35:23049–23062, 2022.
- [4] Frederica Damera. The spmd model: Past, present and future. In European Parallel Virtual Machine/Message Passing Interface Users’Group Meeting, pages 1–1. Springer, 2001.
- [5] Tim Dettmers, Ruslan Svirschevski, Vage Egiazarian, Denis Kuznetsov, Elias Frantar, Saleh Ashkboos, Alexander Borzunov, Torsten Hoeffler, and Dan Alistarh. Spqr: A sparse-quantized representation for near-lossless llm weight compression. arXiv preprint arXiv:2306.03078, 2023.
- [6] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. Journal of Machine Learning Research, 23(120):1–39, 2022.
- [7] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. Training compute-optimal large language models. arXiv preprint arXiv:2203.15556, 2022.
- [8] Ziheng Jiang, Haibin Lin, Yinmin Zhong, Qi Huang, Yangrui Chen, Zhi Zhang, Yanghua Peng, Xiang Li, Cong Xie, Shibiao Nong, et al. {MegaScale}: Scaling large language model training to more than 10,000 {GPUs}. In 21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24), pages 745–760, 2024.
- [9] In-Jun Jung, Gyeongrok Yang, Jaeha Min, and Joo-Young Kim. Hlx: A unified pipelined architecture for optimized performance of hybrid transformer-mamba language models. In Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture, pages 461–475, 2025.

-
- [10] Huaicheng Li, Daniel S Berger, Lisa Hsu, Daniel Ernst, Pantea Zardoshti, Stanko Novakovic, Monish Shah, Samir Rajadnya, Scott Lee, Ishwar Agarwal, et al. Pond: Cxl-based memory pooling systems for cloud platforms. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, Volume 2, pages 574–587, 2023.
 - [11] H. Liao. Ub-mesh: A new interconnection technology for large ai supernodes. In *IEEE Hot Chips*, 2025.
 - [12] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.
 - [13] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019.
 - [14] Dheevatsa Mudigere, Yuchen Hao, Jianyu Huang, Zhihao Jia, Andrew Tulloch, Srinivas Sridharan, Xing Liu, Mustafa Ozdal, Jade Nie, Jongsoo Park, et al. Software-hardware co-design for fast and scalable training of deep learning recommendation models. In *Proceedings of the 49th Annual International Symposium on Computer Architecture*, pages 993–1011, 2022.
 - [15] Darpan Pandey, Kamal Niwaria, and Bharti Chourasia. Machine learning algorithms: a review. *Mach. Learn*, 6(2), 2019.
 - [16] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32, 2019.
 - [17] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. Zero: memory optimizations toward training trillion parameter models. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, SC ’20. IEEE Press, 2020.
 - [18] Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. Deepspeed: System optimizations enable training deep learning models with over 100 billion parameters. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*, pages 3505–3506, 2020.
 - [19] Jie Ren, Samyam Rajbhandari, Reza Yazdani Aminabadi, Olatunji Ruwase, Shuangyan Yang, Minjia Zhang, Dong Li, and Yuxiong He. {Zero-offload}: Democratizing {billion-scale} model training. In *2021 USENIX Annual Technical Conference (USENIX ATC 21)*, pages 551–564, 2021.
 - [20] Xiaozhe Ren, Pingyi Zhou, Xinfan Meng, Xinjing Huang, Yadao Wang, Weichao Wang, Pengfei Li, Xiaoda Zhang, Alexander Podolskiy, Grigory Arshinov, et al. Pangu- Σ : Towards trillion parameter language model with sparse heterogeneous computing. *arXiv preprint arXiv:2303.10845*, 2023.
 - [21] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.
 - [22] Quan Sun, Yufeng Cui, Xiaosong Zhang, Fan Zhang, Qiyong Yu, Yuezhe Wang, Yongming Rao, Jingjing Liu, Tiejun Huang, and Xinlong Wang. Generative multimodal models are in-context learners. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 14398–14409, 2024.
 - [23] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
 - [24] Qiyong Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, et al. Dapo: An open-source llm reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.
 - [25] Xia Zhao, Limin Wang, Yufei Zhang, Xuming Han, Muhammet Deveci, and Milan Parmar. A review of convolutional neural networks in computer vision. *Artificial Intelligence Review*, 57(4):99, 2024.
 - [26] P. Zuo et al. Serving large language models on huawei cloudmatrix384. *arXiv preprint arXiv:2506.12708*, 2025.